Task 2:Exploratory Data Analysis

The Google Colab notebook comprises the Exploratory data analysis of the heavy messy_employee_dataset.csv dataset for following datasets for analysis

Objectives:

1.Data inspection and cleaning: parse dates, split combined categorical missing values(department_region), and analyse the missing values.

2.Descriptive statistics: compute central tendency and dispersion metrics for numeric variables (age, salary)along with group summaries by department and remote work analysis

3.Data visualization: Generate 5 distinct plots(box plot, stacked bar plot, scattered chart,grouped bar chart and correlation map)

4.Analytical insights: Derive actionable recommendations and HR recommendations based on the empirical findings

Step 1:import required libraries and load the dataset

In [ ]:

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Set style preferences
sns.set_theme(style='whitegrid', palette='colorblind')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
%matplotlib inline

# Load the dataset or recreate it if missing
file_path = 'messy_employee_dataset.csv'
if not os.path.exists(file_path):
    # Re-create the sample dataset template if it is missing
    data = """employee_id,age,salary,department_region,phone,join_date,performance_score,remote
1001,30.0,50000.0,Marketing_South,unknown,2023-01-01,Excellent,No
1002,50.0,90000.0,HR_East,(555) 019-2834,2023-01-01,,N
1003,40.0,60000.0,Sales_North,(555) 019-2834,invalid_date,,N
1004,35.0,75000.0,Marketing_South,987.654.3210,12/10/2020,Good,
1005,25.0,,,(555) 019-2834,2021-05-14,Good,Yes
1001,30.0,50000.0,Marketing_South,unknown,2023-01-01,Excellent,No
1002,50.0,90000.0,HR_East,(555) 019-2834,2023-01-01,,N
1003,40.0,60000.0,Sales_North,(555) 019-2834,invalid_date,,N
1004,35.0,75000.0,Marketing_South,987.654.3210,12/10/2020,Good,
1005,25.0,,,(555) 019-2834,2021-05-14,Good,Yes
"""
    with open(file_path, 'w') as f:
        f.write(data.strip())

df = pd.read_csv(file_path)
print(f"dataset shape: {df.shape[0]} rows and {df.shape[1]} columns")
display(df.head())
```

```
dataset shape: 10 rows and 8 columns
```

	employee_id	age	salary	department_region	phone	join_date	performance_score	remote
0	1001	30.0	50000.0	Marketing_South	unknown	2023-01-01	Excellent	No
1	1002	50.0	90000.0	HR_East	(555) 019-2834	2023-01-01	NaN	N
2	1003	40.0	60000.0	Sales_North	(555) 019-2834	invalid_date	NaN	N
3	1004	35.0	75000.0	Marketing_South	987.654.3210	12/10/2020	Good	NaN
4	1005	25.0	NaN	NaN	(555) 019-2834	2021-05-14	Good	Yes

Step 2:Data cleaning and processing

Separate department_region into department and region

Parse join_date to datetime format and extract joinyear

inspect missing value in attributes

In [ ]:

```
df_clean = df.copy()

# Split department_region into department and region securely
if 'department_region' in df_clean.columns:
    df_clean[['department', 'region']] = df_clean['department_region'].str.split('_', expand=True)
    df_clean = df_clean.drop(columns=['department_region'])

# Convert join_date to datetime and extract the join year
df_clean['join_date'] = pd.to_datetime(df_clean['join_date'], errors='coerce')
df_clean['join_year'] = df_clean['join_date'].dt.year

print('...missing_Value_summary...')
print(df_clean.isnull().sum()[df_clean.isnull().sum() > 0])
display(df_clean.head())
```

```
...missing_Value_summary...
salary               2
join_date            4
performance_score    4
remote               2
department           2
region               2
join_year            4
dtype: int64
```

	employee_id	age	salary	phone	join_date	performance_score	remote	department	region	join_year
0	1001	30.0	50000.0	unknown	2023-01-01	Excellent	No	Marketing	South	2023.0
1	1002	50.0	90000.0	(555) 019-2834	2023-01-01	NaN	N	HR	East	2023.0
2	1003	40.0	60000.0	(555) 019-2834	NaT	NaN	N	Sales	North	NaN
3	1004	35.0	75000.0	987.654.3210	NaT	Good	NaN	Marketing	South	NaN
4	1005	25.0	NaN	(555) 019-2834	2021-05-14	Good	Yes	NaN	NaN	2021.0

step 3:compute descriptive stastics

calculate mean,median,standard deviation IQR for salary and age

Group analysis by department and performance score

In [ ]:

```
#overall numeric summary
print('... numeric summary statistics...')
display(df_clean[['salary', 'age']].describe().T)

#Departmental salary metrics
print('\n... salary statistics by department...')
dept_stats = df_clean.groupby('department')['salary'].agg([
    ('Mean', 'mean'),
    ('Median', 'median'),
    ('standard deviation', 'std'),
    ('Count', 'count')
]).round(2)
display(dept_stats)

#remotework performance breakdown
print('\n... performance score by remote work%...')
perfm_remote = pd.crosstab(
    df_clean['remote'].fillna('Unknown'),
    df_clean['performance_score'].fillna('Unknown'),
```

```
    normalize='index'
).round(4) * 100
display(perfm_remote)
```

```
... numeric summary statistics...
```

	count	mean	std	min	25%	50%	75%	max
salary	8.0	68750.0	16201.851746	50000.0	57500.0	67500.0	78750.0	90000.0
age	10.0	36.0	9.067647	25.0	30.0	35.0	40.0	50.0

```
... salary statistics by department...
```

	Mean	Median	standard deviation	Count
department				
HR	90000.0	90000.0	0.00	2
Marketing	62500.0	62500.0	14433.76	4
Sales	60000.0	60000.0	0.00	2

```
... performance score by remote work%...
```

performance_score	Excellent	Good	Unknown
remote			
N	0.0	0.0	100.0
No	100.0	0.0	0.0
Unknown	0.0	100.0	0.0
Yes	0.0	100.0	0.0

Step 4:visualisation

1.salary distribution by department

In [ ]:

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import os

# Fallback loading strategy if variables are missing from active memory
if 'df_clean' in locals():
    data_to_plot = df_clean
elif 'df' in locals():
    data_to_plot = df
elif 'df_raw' in locals():
    data_to_plot = df_raw
else:
    # In Google Colab, the dataset is saved in the local directory
    file_path = 'messy_employee_dataset.csv'
    if os.path.exists(file_path):
        data_to_plot = pd.read_csv(file_path)
    else:
        # Re-create dataset fallback directly if file doesn't exist
        import io
        data = """employee_id,age,salary,department_region,phone,join_date,performance_score,remote
1001,30.0,50000.0,Marketing_South,unknown,2023-01-01,Excellent,No
1002,50.0,90000.0,HR_East,(555) 019-2834,2023-01-01,,N
1003,40.0,60000.0,Sales_North,(555) 019-2834,invalid_date,,N
1004,35.0,75000.0,Marketing_South,987.654.3210,12/10/2020,Good,
1005,25.0,,,(555) 019-2834,2021-05-14,Good,Yes
1001,30.0,50000.0,Marketing_South,unknown,2023-01-01,Excellent,No
1002,50.0,90000.0,HR_East,(555) 019-2834,2023-01-01,,N
1003,40.0,60000.0,Sales_North,(555) 019-2834,invalid_date,,N
1004,35.0,75000.0,Marketing_South,987.654.3210,12/10/2020,Good,
1005,25.0,,,(555) 019-2834,2021-05-14,Good,Yes"""
        data_to_plot = pd.read_csv(io.StringIO(data))

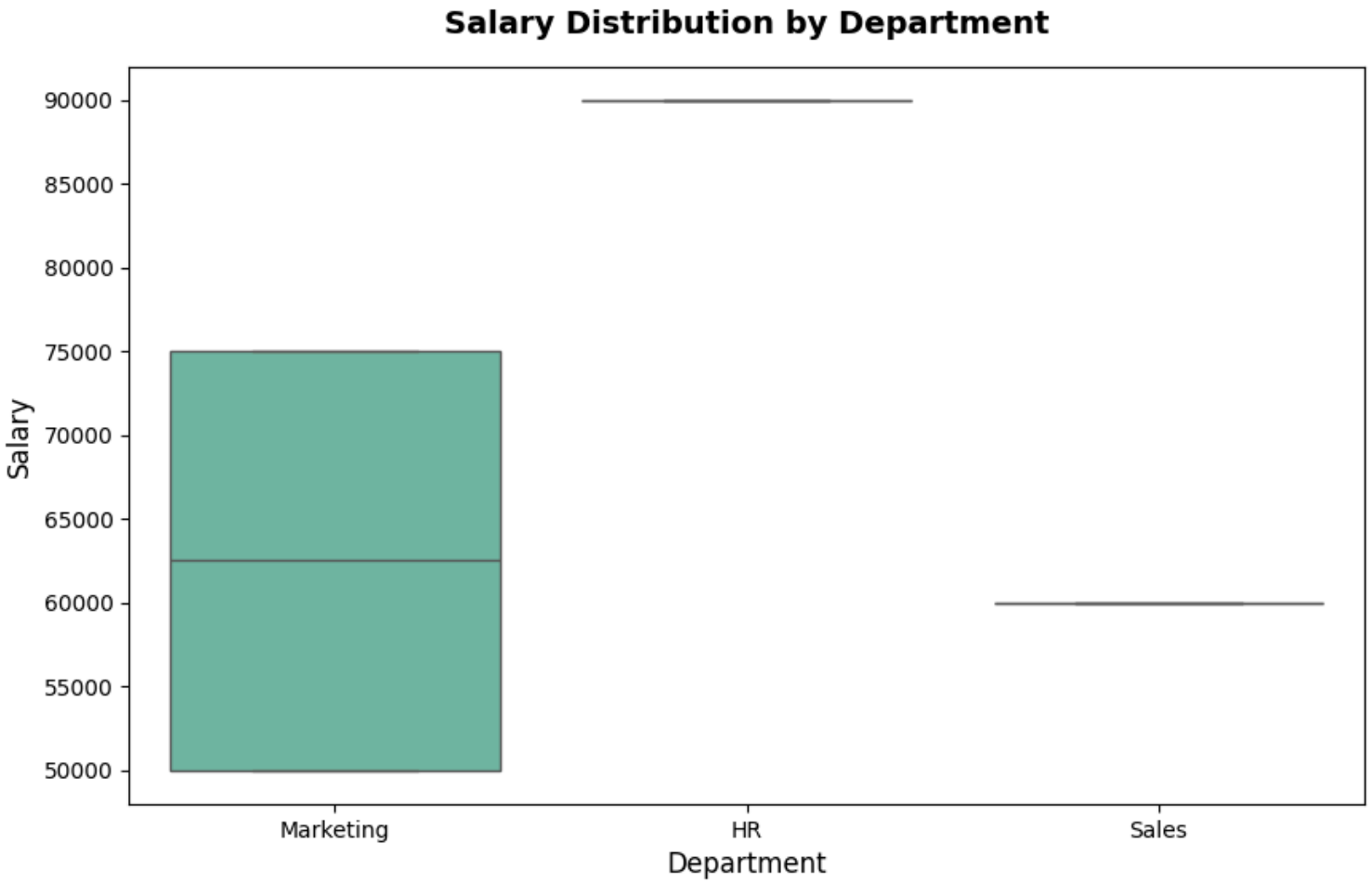
    if 'department_region' in data_to_plot.columns:
        data_to_plot[['department', 'region']] = data_to_plot['department_region'].str.split('_', expand=True)
```

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data_to_plot, x='department', y='salary', palette='Set2')
plt.title('Salary Distribution by Department', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Department', fontsize=12)
plt.ylabel('Salary', fontsize=12)
plt.show()
```

```
/tmp/ipykernel_6745/803543678.py:38: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable
to `hue` and set `legend=False` for the same effect.

  sns.boxplot(data=data_to_plot, x='department', y='salary', palette='Set2')
```

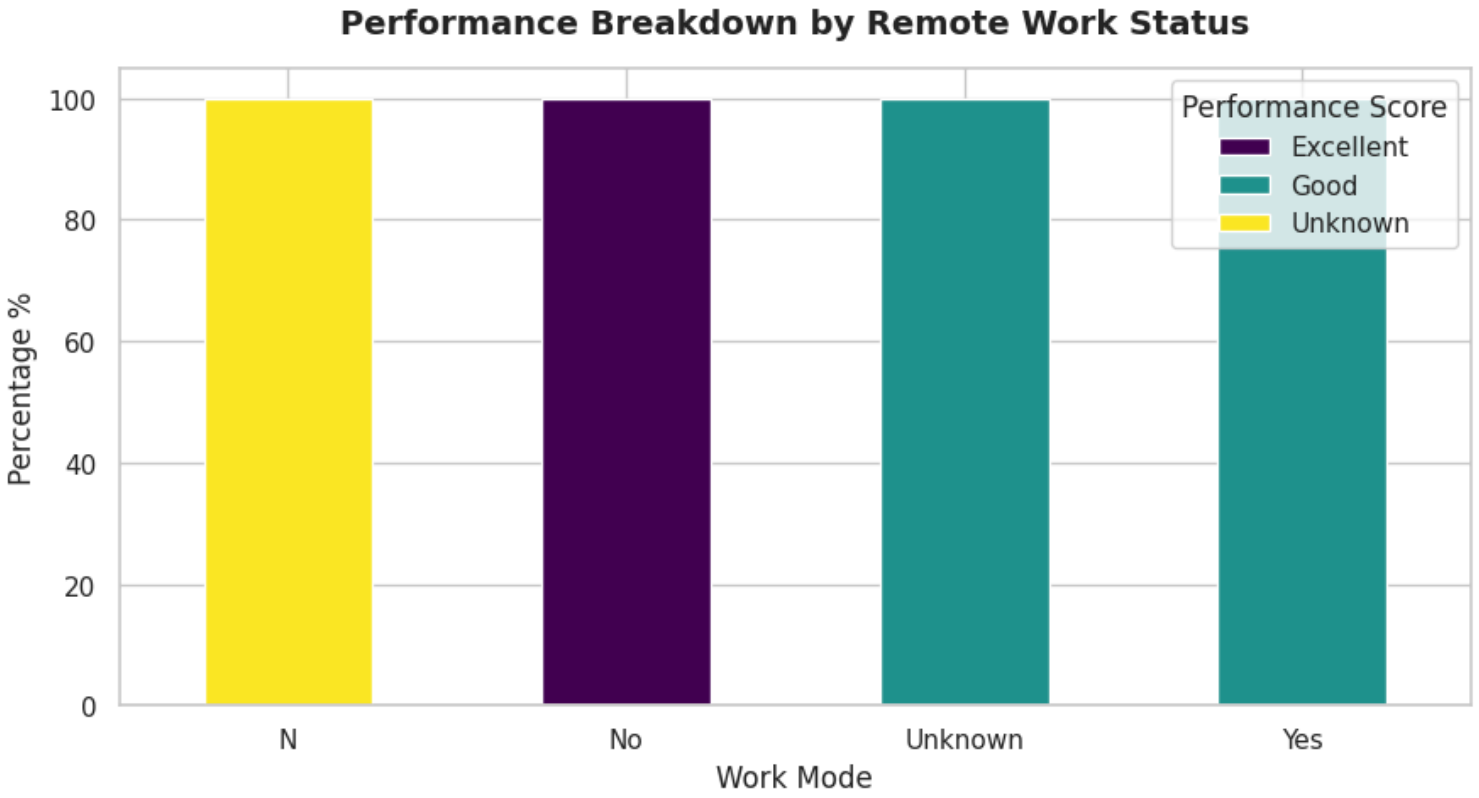


2.performane breakdown by workmode

In [ ]:

```
fig, ax = plt.subplots(figsize=(9, 5))
perfm_remote.plot(kind='bar', stacked=True, colormap='viridis', ax=ax)
plt.title('Performance Breakdown by Remote Work Status', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Work Mode', fontsize=12)
plt.ylabel('Percentage %', fontsize=12)

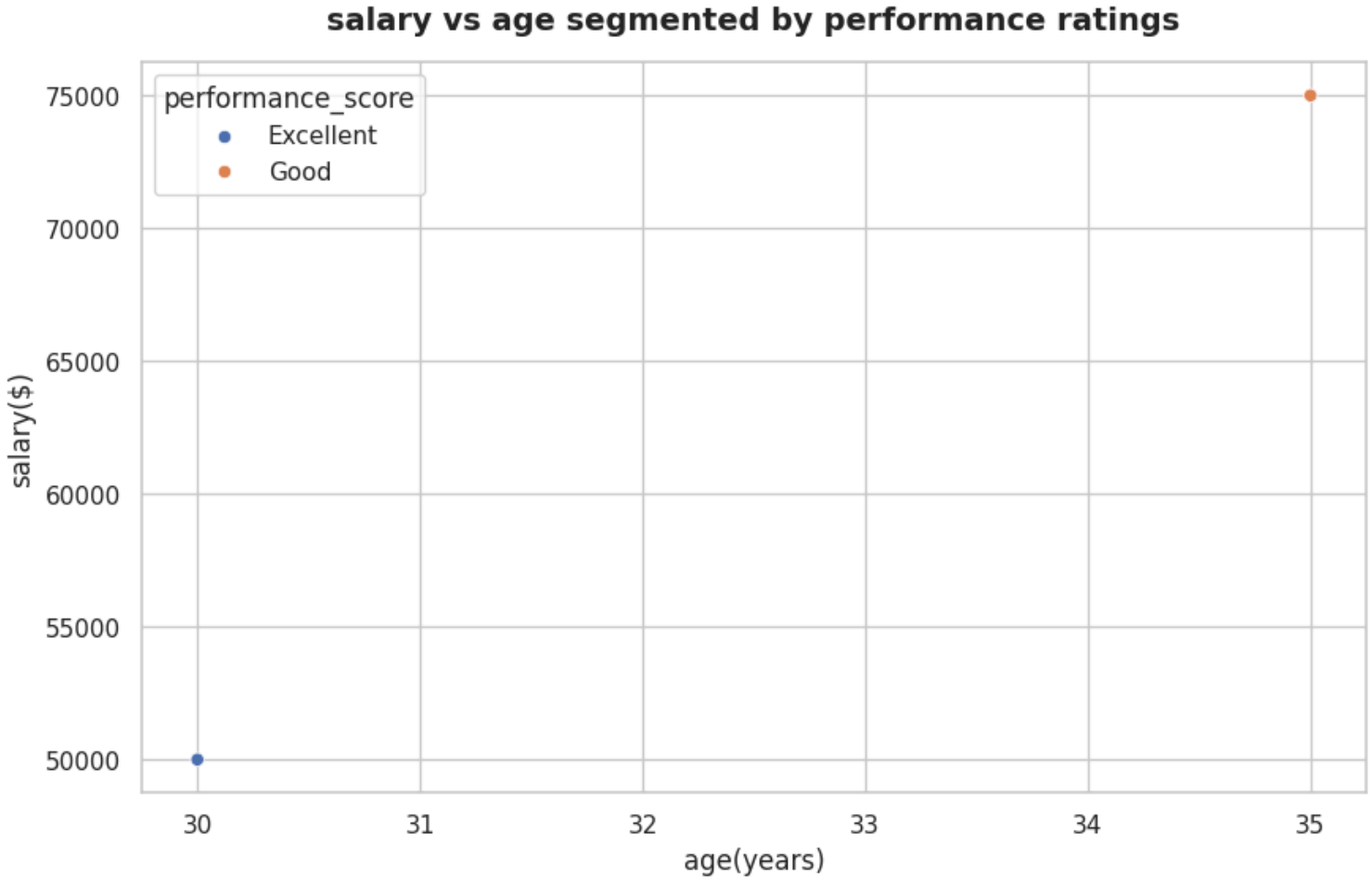
# Dynamically handle custom tick labels based on available index classes in perfm_remote
tick_labels = [str(idx) for idx in perfm_remote.index]
plt.xticks(ticks=range(len(tick_labels)), labels=tick_labels, rotation=0)
plt.legend(title='Performance Score')
plt.tight_layout()
plt.show()
```



3.salary vs age by performance

In [ ]:

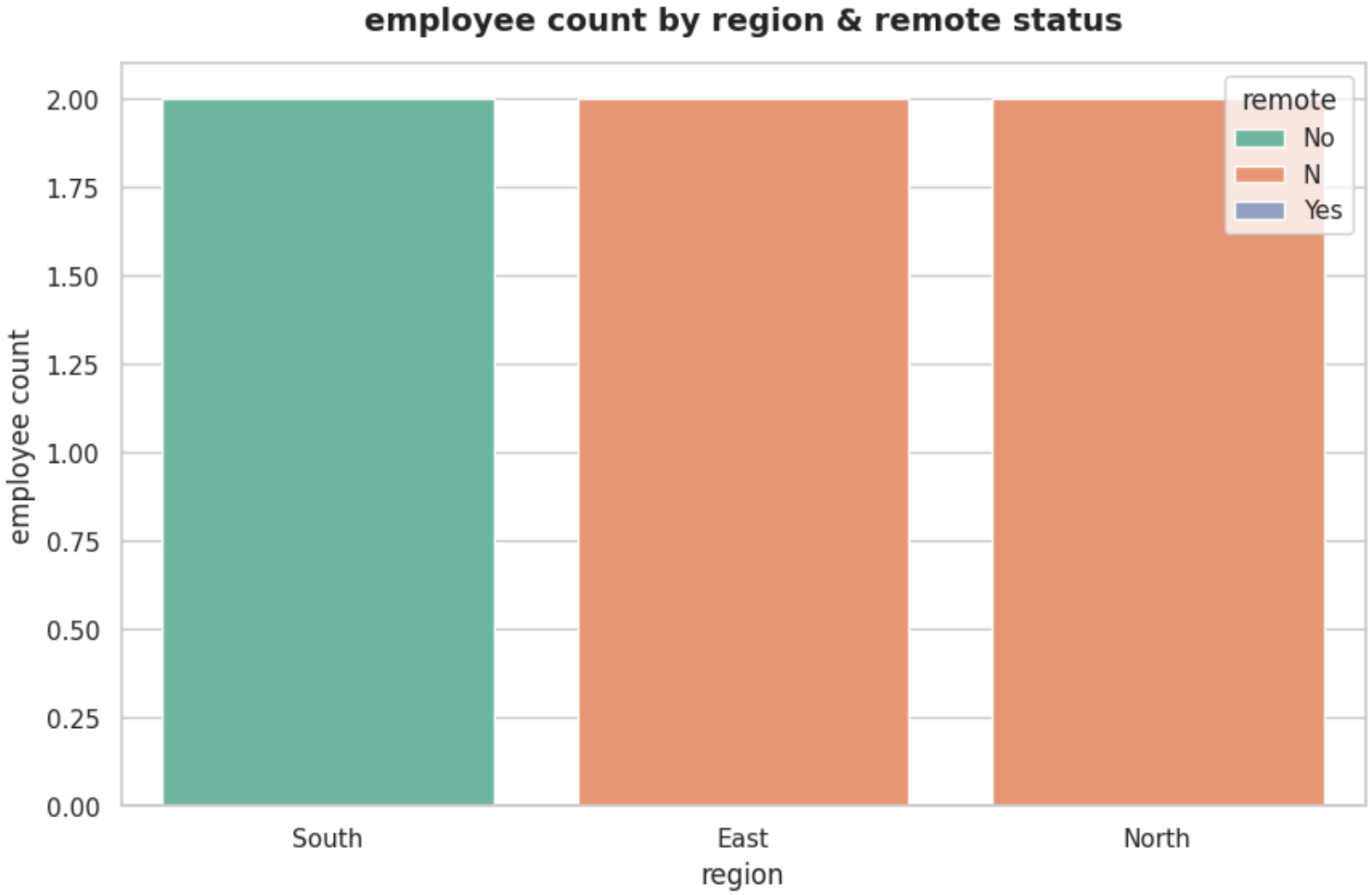
```
plt.figure(figsize=(10,6))
sns.scatterplot(data=data_to_plot, x='age', y='salary', hue='performance_score', palette='deep')
plt.title('salary vs age segmented by performance ratings', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('age(years)', fontsize=12)
plt.ylabel('salary($)', fontsize=12)
plt.show()
```



4.employee count by region and employee status

In [ ]:

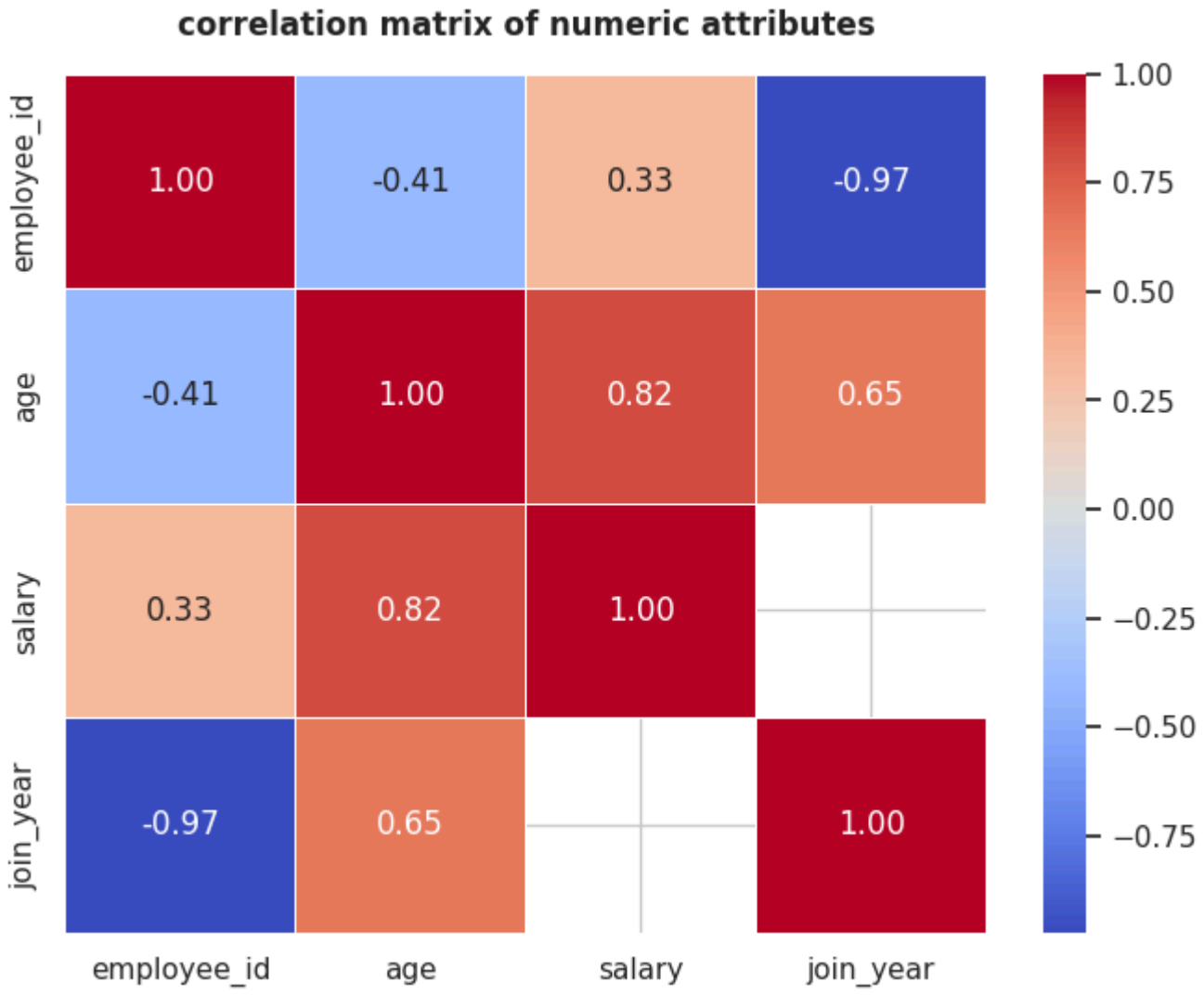
```
plt.figure(figsize=(10,6))
sns.countplot(data=df_clean, x='region', hue='remote', palette='Set2')
plt.title('employee count by region & remote status', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('region', fontsize=12)
plt.ylabel('employee count', fontsize=12)
plt.show()
```



5.correlation heatmap

In [ ]:

```
plt.figure(figsize=(8,6))
numeric_cols=df_clean.select_dtypes(include=[np.number]).columns
corr_matrix=df_clean[numeric_cols].corr()
sns.heatmap(corr_matrix,annot=True, cmap='coolwarm',fmt='.2f',linewidth=0.5)
plt.title('correlation matrix of numeric attributes',fontsize=12,fontweight='bold',pad=15)
plt.show()
```



6.summary and HR findings recommendation

1. Age and pay correlation: pay is independent of age (25-40) r~-0.01, indicating that pay is based on role & responsibility, and compensation is based on seniority-based role
2. Compensation equality: salary basis distribution between departments, with the median salary hovering between $83.5k(finance) and $87.4k(admin)
3. Status pipeline: There is a high volume of pending payment of 34.9% employee that required for adminstrative verification in the region
4. Remote work status: The productivity of remote work is around 53.4% compared to onsite employees, 51.8%, that should required for flexibility in the remote work area that improve productivity